

UsuariosAPI

Documentação Técnica e Guia de Aprovação

Arquitetura, execução passo a passo, contrato REST, banco de dados, segurança, testes, riscos e publicação no GitHub.

VERSÃO	1.0
REVISÃO	22/06/2026
STACK	.NET 8 + SQL Server + Docker
STATUS	Pronto para análise e GitHub

O projeto está funcional e apto para avaliação local e publicação do código no GitHub. A exposição da aplicação à internet ainda depende de autenticação, rate limiting, HTTPS, privilégio mínimo do SQL Server e logs de auditoria.

BUILD	TESTES	NUGET	DOCKER
Aprovado	29 / 29	0 vulneráveis	Healthy

Mapa do documento

- 1-6 Escopo, arquitetura, camadas e tecnologias
- 7-10 Banco de dados e execução
- 11-14 API, validações, erros e Swagger
- 15-18 Segurança, Docker, testes e avaliação
- 19-22 Problemas, aprovação, GitHub e parecer final

DECISÃO RECOMENDADA

Aprovar para demonstração e GitHub. Condicionar o deploy público ao pacote de segurança pendente.

Resultado visual do site

Tela Razor Pages/C# acessada em <http://localhost:8080/site>, mostrando o gerenciamento de usuários integrado ao SQL Server.


USUARIOSAPI

Gerenciamento de usuários

Tela em Razor Pages/C# para testar cadastro, listagem, busca por ID, atualização e exclusão.

POST E PUT

Cadastrar usuário

Nome

Sobrenome

E-mail

Gênero

Data de nascimento (opcional)

Selecione

▼



Cadastrar

Limpar

GET POR ID

Buscar usuário

ID do usuário

Buscar

Informe um ID para consultar um usuário específico.

GET, PUT E DELETE

Usuários cadastrados

1 usuário(s) encontrado(s). Página 1 de 1.

Filtrar por nome

Filtrar por e-mail

Página

Tamanho

Aplicar filtros

ID	NOME	E-MAIL	GÊNERO	NASCIMENTO	AÇÕES
14	Victor de Jesus Santos Lise	victordejesus.santo@gmail.com	Masculino	29/03/2006	Editar Apagar

Página anterior

Próxima página

Atualizar lista

A tela permite testar cadastro, listagem, busca por ID, atualização e exclusão sem sair da aplicação .NET.

1. Objetivo deste documento

Este documento reúne as informações necessárias para compreender, executar, testar, avaliar e aprovar o projeto antes da publicação no GitHub.

Ele descreve:

- o que o sistema faz;
- como as camadas estão organizadas;
- como preparar o SQL Server;
- como executar com e sem Docker;
- como usar todos os endpoints;
- como funcionam validações e erros;
- quais controles de segurança já existem;
- quais riscos ainda precisam de decisão;
- como validar e publicar o repositório.

2. Resumo executivo

UsuariosAPI é uma aplicação ASP.NET Core 8 composta por uma API REST de usuários e uma interface web em Razor Pages/C#. A tela permite cadastrar, listar, consultar por ID, atualizar e excluir usuários, usando a mesma camada de aplicação da API.

O sistema utiliza SQL Server, Entity Framework Core, migrations, FluentValidation, Swagger/OpenAPI, xUnit, Moq e Docker. A solução está separada em Domain, Application, Infrastructure e API.

Na última verificação:

- o build terminou sem erros;
- 18 testes unitários passaram;
- 11 testes de integração passaram;
- a auditoria NuGet não encontrou pacotes vulneráveis;
- o container ficou com status **healthy**;
- o container executou com usuário não privilegiado;
- um cadastro real foi criado e removido com sucesso no SQL Server.

3. Escopo funcional

3.1 Incluído

- Página web responsiva em Razor Pages/C# para CRUD visual.
- Cadastro de usuário pela API.
- Listagem paginada com filtros.
- Consulta de usuário por ID.
- Atualização completa.
- Exclusão física.

- Validação de entrada.
- E-mail único.
- Tratamento global de erros.
- Swagger/OpenAPI.
- Health check.
- SQL Server com migrations.
- Docker conectado ao SQL Server do Windows.
- Testes unitários e de integração.
- Cabeçalhos HTTP de segurança.

3.2 Não incluído no estado atual

- Login e autenticação.
- Autorização por perfil.
- Rate limiting.
- Recuperação de senha.
- Soft delete.
- Histórico persistente de operações.
- HTTPS terminado dentro do container.
- Pipeline automático do GitHub Actions.
- Deploy em nuvem.

Esses itens não impedem avaliação local. Autenticação, rate limiting, HTTPS e privilégio mínimo do banco são recomendados antes de exposição pública.

4. Visão da solução

4.1 Endereços

Endereço	Componente	Público alvo
http://localhost:8080/site	Tela Razor Pages de gerenciamento	Usuário final
http://localhost:8080/swagger	Swagger	Desenvolvedor e avaliador
http://localhost:8080/health	Health check	Operação e monitoramento
http://localhost:8080/api/v1/usuarios	API REST	Sistemas clientes

Todos os endereços são rotas da mesma aplicação. Não existem três servidores diferentes.

4.2 Fluxo pela tela Razor Pages

```

Navegador
|
| POST formulário Razor
v
Index.cshtml.cs
|
v
UsuarioService -> FluentValidation
|
v
IUsuarioRepository
|
v
Entity Framework Core
|
v
SQL Server - usuarios_db
    
```

Os endpoints REST continuam disponíveis em `/api/v1/usuarios` e seguem o fluxo `UsuariosController -> UsuarioService -> Repository -> SQL Server`.

4.3 Dependências entre camadas

```

UsuariosAPI.API
|-- UsuariosAPI.Application
`-- UsuariosAPI.Infrastructure

UsuariosAPI.Infrastructure
|-- UsuariosAPI.Application
`-- UsuariosAPI.Domain

UsuariosAPI.Application
`-- UsuariosAPI.Domain

UsuariosAPI.Domain
`-- sem dependências de outros projetos da solução
    
```

5. Organização do código

Camada	Responsabilidade	Exemplos
Domain	Regras e abstrações centrais	<code>Usuario</code> , <code>Genero</code> , <code>IUsuarioRepository</code>
Application	Casos de uso, validação e contratos HTTP internos	<code>UsuarioService</code> , DTOs, validators
Infrastructure	Persistência e detalhes técnicos	<code>AppDbContext</code> , <code>UsuarioRepository</code> , migrations
API	Entrada HTTP e inicialização	Controller, Razor Pages, middleware, Swagger

5.1 Domain

A entidade `Usuario` possui setters privados. A criação ocorre por construtor e a alteração ocorre pelo método `Atualizar`. As datas `CriadoEm` e `AtualizadoEm` são geradas em UTC.

O enum `Genero` aceita:

- `Masculino`;
- `Feminino`;
- `Outro`.

5.2 Application

`UsuarioService` coordena as validações, regras de negócio e chamadas ao repositório. Controllers não acessam diretamente o banco.

DTOs separam o contrato externo da entidade de persistência:

- `CriarUsuarioRequest`;
- `AtualizarUsuarioRequest`;
- `ListarUsuariosRequest`;
- `UsuarioResponse`;
- `PagedResponse<T>`;
- `ApiResponse<T>`.

5.3 Infrastructure

`AppDbContext` configura a tabela, colunas, tipos, obrigatoriedade e índice único do e-mail. `UsuarioRepository` concentra as consultas LINQ e gravações.

5.4 API

`UsuariosController` publica os cinco endpoints REST. `Pages/Index.cshtml` e `Pages/Index.cshtml.cs` implementam a tela web em Razor Pages/C#. `ExceptionHandlerMiddleware` converte exceções em respostas HTTP consistentes para a API. `ServiceCollectionExtensions` centraliza a injeção de dependências.

O diretório `wwwroot` contém apenas arquivos estáticos, como a folha de estilos. A lógica da tela fica em C# no `PageModel`.

6. Tecnologias e versões verificadas

Tecnologia	Versão
.NET / ASP.NET Core	8
Entity Framework Core	8.0.28
EF Core SQL Server	8.0.28
FluentValidation	11.9.2
Swashbuckle.AspNetCore	6.6.2
xUnit	2.9.3
Moq	4.20.72
coverlet.collector	6.0.2
SQL Server Express	Instância local SQLEXPRESS03
Docker	Imagem SDK 8.0.422 e Runtime 8.0.28

7. Banco de dados

7.1 Modelo

Tabela: `dbo.usuarios`

Coluna	Tipo	Regra
<code>id</code>	BIGINT IDENTITY	Chave primária

Coluna	Tipo	Regra
nome	NVARCHAR(100)	Obrigatório
sobrenome	NVARCHAR(100)	Obrigatório
email	NVARCHAR(150)	Obrigatório e único
genero	NVARCHAR(20)	Obrigatório
data_nascimento	DATE	Opcional
criado_em	DATETIME2	Obrigatório, UTC
atualizado_em	DATETIME2	Obrigatório, UTC

Índice único: `idx_usuarios_email`.

7.2 Criação recomendada

O caminho principal é permitir que o Entity Framework aplique a migration versionada na inicialização. O arquivo `database/init.sql` é uma alternativa manual.

7.3 Observação sobre privilégios

No desenvolvimento, o login `usuarios_api` pode usar `db_owner` porque a aplicação executa `MigrateAsync` ao iniciar.

Para produção:

1. aplique migrations com uma credencial administrativa;
2. utilize uma credencial diferente para a aplicação;
3. dê ao usuário de execução somente leitura e escrita nas tabelas necessárias;
4. armazene a senha em um gerenciador de segredos.

8. Preparação completa do SQL Server

8.1 Habilitar TCP/IP

1. Abra SQL Server Configuration Manager.
2. Acesse `SQL Server Network Configuration`.
3. Abra `Protocols for SQLEXPRESS03`.
4. Habilite `TCP/IP`.
5. Abra as propriedades do protocolo.
6. Em `IP Addresses > IPAll`, limpe `TCP Dynamic Ports`.
7. Informe `1433` em `TCP Port`.
8. Reinicie `SQL Server (SQLEXPRESS03)`.

8.2 Confirmar conectividade

```
Test-NetConnection localhost -Port 1433
```

Resultado esperado:

```
TcpTestSucceeded : True
```

8.3 Habilitar modo misto

No SSMS, abra as propriedades do servidor, entre em `Security` e marque `SQL Server and Windows Authentication mode`. Reinicie o serviço.

8.4 Criar banco e login de desenvolvimento

```
USE master;
GO

IF DB_ID(N'usuarios_db') IS NULL
    CREATE DATABASE usuarios_db;
GO

IF NOT EXISTS (SELECT 1 FROM sys.sql_logins WHERE name = N'usuarios_api')
    CREATE LOGIN usuarios_api
    WITH PASSWORD = N'TROQUE_POR_UMA_SENHA_FORTE';
GO

USE usuarios_db;
GO

IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name = N'usuarios_api')
    CREATE USER usuarios_api FOR LOGIN usuarios_api;
GO

ALTER ROLE db_owner ADD MEMBER usuarios_api;
GO
```

Não reutilize a senha do exemplo e não registre a senha real no Git.

9. Execução com Docker

9.1 Configurar

```
Set-Location "C:\Users\victo\Documents\UsuariosAPI\UsuariosAPI"
Copy-Item .env.example .env
```

Edite a variável:

```
SQLSERVER_DOCKER_CONNECTION=Server=host.docker.internal,1433;Database=usuarios_db;User Id=usuarios_api;Password=SUA_SENHA;Encrypt=True;TrustServerCertificate=True;MultipleActiveResultSets=True;
```

9.2 Construir e iniciar

```
docker compose up -d --build
docker compose ps
docker compose logs --tail 100 api
```

O Compose inicia somente a API. O SQL Server permanece instalado no Windows.

9.3 Resultado esperado

NAME	SERVICE	STATUS
usuarios_api	api	Up ... (healthy)

9.4 Encerrar

```
docker compose down
```

10. Execução local sem Docker

Configure a conexão com User Secrets:

```
dotnet user-secrets set "ConnectionStrings:DefaultConnection" "Server=localhost\SQLEXPRESS03;Database=usuarios_db;Trusted_Connection=True;Encrypt=True;TrustServerCertificate=True;MultipleActiveResultSets=True;" --project src/UsuariosAPI.API
dotnet run --project src/UsuariosAPI.API
```

O User Secrets é usado somente no desenvolvimento. O `.env` é consumido pelo Docker Compose, não pela aplicação local.

11. Contrato da API

Base URL no Docker: <http://localhost:8080/api/v1>.

11.1 POST /usuarios

Cria um usuário.

Requisição:

```
{
  "nome": "Victor",
  "sobrenome": "Souza",
  "email": "victor@exemplo.com",
  "genero": "Masculino",
  "dataNascimento": "1998-07-14"
}
```

Resposta 201:

```
{
  "sucesso": true,
  "mensagem": "Usuário cadastrado com sucesso.",
  "dados": {
    "id": 1,
    "nome": "Victor",
    "sobrenome": "Souza",
    "email": "victor@exemplo.com",
    "genero": "Masculino",
    "dataNascimento": "1998-07-14T00:00:00",
    "criadoEm": "2026-06-22T22:00:00Z",
    "atualizadoEm": "2026-06-22T22:00:00Z"
  }
}
```

Possíveis respostas: 201, 400, 409, 422 e 500.

11.2 GET /usuarios

Lista usuários com paginação.

Parâmetro	Padrão	Regra
<code>pagina</code>	1	Mínimo 1
<code>tamanhoPagina</code>	10	Entre 1 e 100
<code>nome</code>	vazio	Busca parcial em nome ou sobrenome
<code>email</code>	vazio	Busca parcial em e-mail

Exemplo:

```
GET /api/v1/usuarios?pagina=1&tamanhoPagina=10&nome=Victor
```

Resposta 200:

```
{
  "sucesso": true,
  "mensagem": "Usuários listados com sucesso.",
  "dados": {
    "data": [],
    "pagina": 1,
    "tamanhoPagina": 10,
    "total": 0,
    "totalPaginas": 0
  }
}
```

11.3 GET /usuarios/{id}

Retorna o usuário indicado. O ID deve ser maior que zero.

Possíveis respostas: 200, 404, 422 e 500.

11.4 PUT /usuarios/{id}

Atualiza todos os campos do usuário. O contrato é equivalente ao POST. Por ser PUT, envie o conjunto completo de dados.

Possíveis respostas: 200, 400, 404, 409, 422 e 500.

11.5 DELETE /usuarios/{id}

Exclui fisicamente o usuário.

Resposta de sucesso: 204 sem corpo.

Possíveis respostas: 204, 404, 422 e 500.

12. Validações

Campo	Validação
Nome	Obrigatório e até 100 caracteres
Sobrenome	Obrigatório e até 100 caracteres
E-mail	Obrigatório, formato válido, até 150 caracteres e único
Gênero	Valor válido do enum
Data de nascimento	Opcional e não pode estar no futuro
Página	Maior ou igual a 1
Tamanho da página	Entre 1 e 100
Filtro de nome	Até 100 caracteres
Filtro de e-mail	Até 150 caracteres

O navegador executa validações básicas para melhorar a experiência. A API repete todas as validações necessárias, pois o cliente não é considerado confiável.

13. Tratamento de erros

Formato padronizado:

```
{
  "sucesso": false,
  "mensagem": "Não foi possível processar a requisição.",
  "dados": null,
  "erros": ["E-mail inválido."],
  "traceId": "identificador-da-requisicao"
}
```

Código	Categoria	Exemplo
400	Erro de formato HTTP	JSON malformado ou tipo incorreto
404	Recurso inexistente	ID não encontrado
409	Conflito de negócio	E-mail já cadastrado
422	Validação semântica	Data futura ou nome vazio
500	Erro interno	Falha inesperada

O `traceId` permite correlacionar a resposta com logs. Stack trace, tipo e mensagem técnica aparecem apenas em Development. Em Production, o cliente recebe mensagem genérica.

14. Swagger e documentação OpenAPI

Swagger está disponível em `/swagger` quando `ASPNETCORE_ENVIRONMENT=Development`.

Cada endpoint informa:

- método e rota;
- parâmetros;
- tipo de corpo;
- resposta de sucesso;
- principais códigos de erro;
- schemas dos DTOs.

Em Production, o Swagger não é exposto pelo pipeline atual.

15. Segurança

15.1 Controles implementados

- EF Core e LINQ parametrizam consultas contra SQL Injection.
- DTOs explícitos reduzem mass assignment.
- FluentValidation limita formato e tamanho da entrada.
- A tela Razor usa encoding HTML padrão do ASP.NET Core e não injeta HTML retornado pelo usuário.
- JSON utiliza o serializador padrão do ASP.NET Core.
- Content Security Policy restringe scripts, estilos e conexões do site.
- `X-Content-Type-Options: nosniff`.
- `X-Frame-Options: DENY`.
- `Referrer-Policy: no-referrer`.
- Detalhes internos ocultos em Production.
- `.env` ignorado pelo Git e pelo contexto Docker.
- Container executado como usuário `app`, não root.
- `cap_drop: ALL`.
- `no-new-privileges:true`.
- Sistema de arquivos do container somente leitura.
- Imagens .NET fixadas em versões específicas.
- Auditoria atual do NuGet sem vulnerabilidades conhecidas.

15.2 Controles pendentes para internet

Risco	Situação atual	Recomendação
Acesso administrativo	GET, PUT e DELETE públicos	JWT ou chave de API com autorização
Abuso de cadastro	Sem rate limiting	Limitar POST por IP e janela de tempo
Transporte	HTTP local	HTTPS em IIS, Nginx, nuvem ou proxy reverso
Privilégio SQL	db_owner no desenvolvimento	Conta de migration separada
Auditoria de negócio	Sem histórico persistente	Logs estruturados para criar, alterar e excluir
Dados pessoais	Listagem retorna e-mail e nascimento	Resumo público e detalhes somente autorizados

Conclusão de segurança: adequado para avaliação local. Não classificado como pronto para internet até tratar autenticação, rate limiting, HTTPS e privilégio SQL.

16. Docker e operação

O Dockerfile possui múltiplos estágios:

1. restauração e build com SDK;
2. publicação Release;
3. imagem final apenas com runtime.

O `.dockerignore` impede envio de `.env`, Git, builds, testes e arquivos de trabalho ao contexto.

O health check consulta `/health`. O Compose adiciona diretório `/tmp` temporário porque o restante do sistema de arquivos é somente leitura.

17. Testes e qualidade

Comandos:

```
dotnet test UsuariosAPI.sln --configuration Release
dotnet list UsuariosAPI.sln package --vulnerable --include-transitive
docker compose config --quiet
```

Testes unitários cobrem serviço, validators e middleware. Testes de integração cobrem CRUD, erros, paginação, site e cabeçalhos de segurança.

Último resultado conhecido:

Suíte	Aprovados	Falhas
Unitários	18	0
Integração	11	0
Total	29	0

Limitação: os testes de integração usam EF Core InMemory. Um próximo refinamento pode acrescentar testes relacionais com SQL Server de teste ou Testcontainers.

18. Avaliação pelos critérios solicitados

Critério	Estado	Evidência ou pendência
Design RESTful	Atendido	Métodos, rotas plurais, versão v1 e status adequados
Tratamento de erros	Atendido	Middleware, mensagens padronizadas e traceId
Documentação	Atendido	Swagger, README, documento técnico e PDF
Validações	Atendido	FluentValidation e respostas amigáveis
Padrões de código	Atendido	Camadas, DI, DTOs, services e repositories
Testes	Atendido	29 testes; recomendado acrescentar banco relacional real
SQL Injection / XSS	Atendido no escopo	EF parametrizado, CSP e saída sem HTML dinâmico
Escalabilidade	Parcial	Aplicação stateless; migration automática deve ser separada em produção
Manutenibilidade	Atendido	Interfaces, separação e responsabilidades claras
Deploy e execução	Atendido localmente	Docker, health check e instruções; falta destino de produção
Banco de dados	Atendido	Mapeamento, migration, PK e índice único

19. Solução de problemas

19.1 Erro de certificado

Para ambiente local, use `Encrypt=True;TrustServerCertificate=True`. Em produção, configure certificado emitido por autoridade confiável.

19.2 Erro 26

Verifique:

- instância `SQLEXPRESS03`;
- serviço iniciado;
- TCP/IP habilitado;
- porta fixa 1433;
- firewall;
- modo misto;
- `Test-NetConnection`.

19.3 Erro 911 - banco inexistente

Execute primeiro `CREATE DATABASE usuarios_db`. Somente depois execute `USE usuarios_db`.

19.4 Variável Docker ausente

Se o Compose informar que `SQLSERVER_DOCKER_CONNECTION` está ausente, crie `.env` a partir de `.env.example`.

19.5 Login falhou

Confirme a senha, habilite o login, crie o usuário dentro de `usuarios_db` e conceda a função adequada.

20. Checklist de aprovação

Funcional

- [] Abrir o site em `http://localhost:8080/site`.

- [] Cadastrar um usuário válido.
- [] Confirmar mensagem de sucesso.
- [] Tentar cadastrar o mesmo e-mail e confirmar 409.
- [] Testar GET, PUT e DELETE no Swagger.
- [] Confirmar paginação e filtros.

Técnico

- [] Executar os 29 testes.
- [] Confirmar zero vulnerabilidades no NuGet.
- [] Confirmar container **healthy**.
- [] Confirmar que **.env** está ignorado.
- [] Conferir migrations e tabela no SSMS.
- [] Ler as pendências de segurança para publicação.

Documentação

- [] Conferir README.
- [] Conferir este documento.
- [] Conferir o PDF renderizado.
- [] Escolher licença do repositório.

21. Publicação no GitHub

21.1 Conferência antes do commit

```
dotnet test UsuariosAPI.sln --configuration Release
dotnet list UsuariosAPI.sln package --vulnerable --include-transitive
docker compose config --quiet
git status --short
git check-ignore .env
```

Não prossiga se **.env**, senha, token ou connection string real aparecerem entre os arquivos preparados.

21.2 Primeiro envio

```
git init
git add .
git status
git commit -m "feat: adiciona API de usuarios documentada"
git branch -M main
git remote add origin https://github.com/SEU-USUARIO/UsuariosAPI.git
git push -u origin main
```

21.3 Sugestão de descrição do repositório

API REST em .NET 8 para gerenciamento de usuários, com SQL Server, Clean Architecture, Swagger, validações, testes e Docker.

22. Parecer final

O sistema está pronto para demonstração, avaliação técnica e publicação do código-fonte no GitHub, desde que o arquivo **.env** e demais segredos não sejam enviados.

Para publicar a aplicação na internet e receber tráfego real, a aprovação deve ficar condicionada a autenticação dos endpoints administrativos, rate limiting, HTTPS, privilégio mínimo do SQL Server e logs de auditoria.